UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA — DIVISIÓN DE INGENIERÍA ELÉCTRICA

Práctica 10: Excepciones y Jerarquía de Bloques Catch

Demostración experimental del orden de captura sobre un arreglo

Equipo 2 — Grupo 7

Integrantes: Alan, Zoé, Luis, Joseph, Hanniel, Alex

Noviembre 2026

Fundamento Conceptual del Manejo de Excepciones

La regla canónica de orden de captura polimórfica en Java

Naturaleza de las Excepciones:

- Separan la lógica habitual de situaciones anómalas.
- Descienden de `Throwable`:
  - **`Error`:** Fallos críticos irrecuperables de la JVM.
  - **`Exception`:** Contingencias previsibles y recuperables.
- **`RuntimeException`:** Defectos de programación o manipulación incorrecta de datos en memoria.

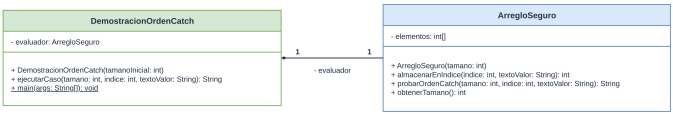
Principio de Especificidad (JLS §14.20):

- La evaluación de cláusulas `catch` es estrictamente secuencial descendente.
- **Regla obligatoria:** Ordenar de subclases particulares a superclases generales.
- **Riesgo de inversión:** Una superclase previa enmascara a sus subclases, produciendo error de compilación por código inalcanzable.

Arquitectura del Sistema: Diagrama de Clases Global

Relación de composición 1 a 1 entre el controlador y la entidad evaluadora

Diagrama de Clases UML — Arquitectura Global con Composición



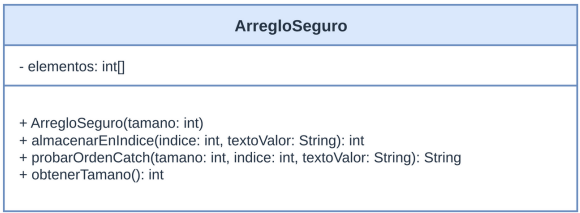
Decisiones de Diseño:

- **Composición Fuerte (1 a 1):** `DemostracionOrdenCatch` es propietaria exclusiva del objeto evaluador.
- **Ciclo de Vida Confinado:** La entidad evaluadora nace en el constructor del controlador y depende de su existencia.
- **Encapsulamiento:** Estado privado con interfaces públicas controladas.

Diseño de la Entidad: Clase ArregloSeguro

Encapsulamiento del arreglo primitivo y evaluación de excepciones

Diagrama de Clase — ArregloSeguro



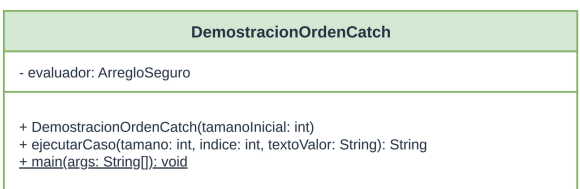
Responsabilidades Clave:

- **Encapsulamiento de Arreglo:** Protege el contenedor primitivo `elementos`.
- **Constructor Parametrizado:** Controla la dimensión e identifica tamaños ilícitos.
- **Inserción Controlada:** Convierte texto a valor primitivo y valida el índice destino.
- **Evaluador de Captura:** Orquesta la secuencia de bloques `catch` de lo específico a lo genérico.

Diseño del Controlador: Clase DemostracionOrdenCatch

Coordinación del flujo interactivo y ejecución automatizada de pruebas

Diagrama de Clase — DemostracionOrdenCatch

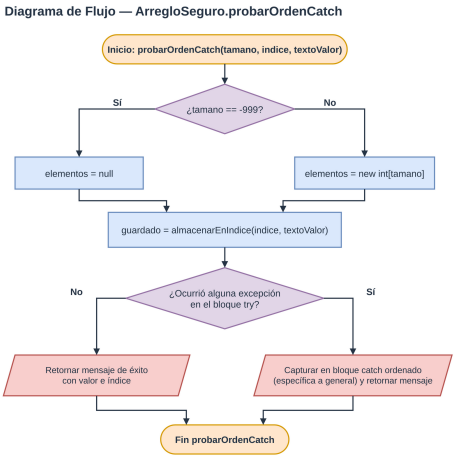


Responsabilidades Clave:

- **Punto de Entrada:** Método estático principal (`main`) para iniciar la ejecución.
- **Interacción por Consola:** Lectura guiada de tamaño, índice y valor de entrada.
- **Despacho de Casos:** Invoca al evaluador y formatea el reporte de salida.
- **Batería Automática:** Induce consecutivamente casos de índice, tamaño, formato y referencia.

Dinámica del Evaluador: Flujo de ArregloSeguro

Evaluación estructurada y secuencia ordenada de captura de excepciones



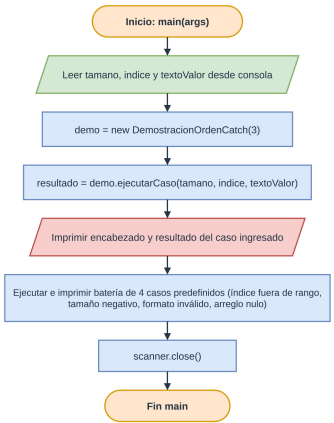
Secuencia de Decisiones:

- **Inicialización:** Configura caso de referencia nula si el tamaño recibido es bandera.
- **Ejecución Segura:** Intenta reserva, conversión numérica e indexación.
- **Capturas Específicas Prioritarias:**
  1. Índice fuera de rango (`ArrayIndexOutOfBounds`)
  2. Tamaño negativo (`NegativeArraySize`)
  3. Formato numérico (`NumberFormat`)
  4. Referencia nula (`NullPointer`)
- **Captura General:** Contención con clases base (`Exception`, `Throwable`) sin ocultar causas particulares.

Dinámica del Controlador: Flujo de DemostracionOrdenCatch

Lectura de parámetros por consola y ejecución secuencial de pruebas

Diagrama de Flujo — DemostracionOrdenCatch.main

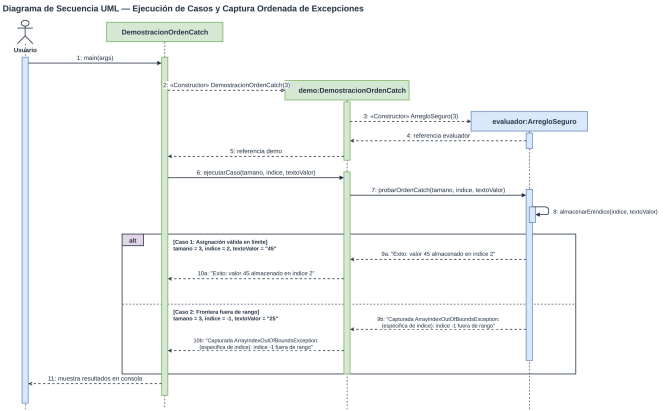


Secuencia Operativa:

- **Inicialización:** Conexión a la consola y creación del controlador.
- **Caso de Usuario:** Lectura de parámetros dinámicos y envío al evaluador.
- **Batería de Pruebas Automáticas:**
  - Índice desbordado.
  - Dimensión negativa.
  - Formato no convertible.
  - Referencia no inicializada.
- **Liberación:** Cierre de recursos de entrada y reporte en consola.

Interacción Temporal: Diagrama de Secuencia

Colaboración entre actor, clase principal, instancia coordinadora y entidad evaluadora



Secuencia de Mensajes:

- **Arranque:** Invocación del método estático principal.
- **Composición:** Construcción de la instancia coordinadora y de la entidad de arreglo.
- **Delegación:** Envío de parámetros y autollamada anidada de inserción.
- **Bifurcación Alternativa:**
  - **Caso 1:** Inserción exitosa.
  - **Caso 2:** Captura unívoca por manejador específico de índice.

Conclusiones y Principios de Diseño

Lecciones fundamentales de la gestión de excepciones en programación orientada a objetos

- **Polimorfismo y Jerarquía:** La captura de excepciones responde rigurosamente al principio de sustitución de Liskov. El orden descendente de especificidad es indispensable para un manejo determinista y sin ambigüedades.
- **Diferenciación Arquitectónica:** Los fallos críticos de sistema (`Error`) deben segregarse categóricamente de las anomalías recuperables (`Exception`), asegurando la estabilidad del entorno de ejecución.
- **Robustez sobre Estructuras de Datos:** Las estructuras de almacenamiento indexadas exigen validación preventiva de invariantes combinada con bloques de contingencia tipados.
- **Diseño Profesional:** La anticipación de excepciones previene fallos silenciosos y garantiza la tolerancia a fallos tanto en servicios de cómputo como en interfaces gráficas interactivas.